

Cloud, Securite et Terraform

7 Mini Projets avec LocalStack

Debutants Terraform | Simulation locale : Docker + LocalStack | Aucun compte cloud requis

01

BaaS

02

FaaS

03

IAM

04

Stockage

05

Vulnerabilite

06

Scalabilite

07

Securite
complete

Niveau : debutants Terraform | Outil : LocalStack (Docker) | 30 min theorie + 30 min pratique par projet

Introduction : Environnement de travail commun

Tous les mini-projets utilisent le meme environnement local. Aucun compte AWS reel n'est necessaire.

Installation (a faire une seule fois)

```
# === Setup initial ===

# 1. Docker Desktop (https://www.docker.com/products/docker-desktop)
# 2. Terraform
brew install terraform      # macOS
choco install terraform     # Windows
# 3. LocalStack et outils AWS local
pip install localstack awscli-local
# 4. Demarrer LocalStack
localstack start
# 5. Verifier
awslocal s3 ls  # doit repondre sans erreur
```

Fichier main.tf commun a tous les projets

```
# === main.tf - Provider LocalStack ===

terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region                = "us-east-1"
  access_key            = "test"
  secret_key            = "test"
  skip_credentials_validation = true
  skip_metadata_api_check  = true
  skip_requesting_account_id = true
  endpoints {
    iam          = "http://localhost:4566"
    s3           = "http://localhost:4566"
    lambda       = "http://localhost:4566"
    dynamodb    = "http://localhost:4566"
    kms          = "http://localhost:4566"
    cloudwatch   = "http://localhost:4566"
    logs         = "http://localhost:4566"
    rds          = "http://localhost:4566"
    ec2          = "http://localhost:4566"
    sns          = "http://localhost:4566"
    sqs          = "http://localhost:4566"
    autoscaling  = "http://localhost:4566"
  }
}
```

Workflow Terraform — les 4 commandes essentielles

terraform init	→ Telecharge le provider AWS (a faire 1 fois par dossier)
terraform plan	→ Montre ce qui va etre cree/modifie/supprime (dry-run)
terraform apply	→ Deploie reellement les ressources sur LocalStack
terraform destroy	→ Supprime toutes les ressources du projet

Theorie — Qu'est-ce que le BaaS ?

Le BaaS (Backend as a Service) est un modele cloud ou le fournisseur gere l'integralite du backend : base de donnees, authentication, stockage de fichiers, notifications push. Le developpeur n'ecrit que le code frontend/mobile.

Vous gerez vs Provider	Caracteristiques
Vous gerez	Code frontend, logique metier
Le provider gere	Serveurs, base de donnees, scaling, securite infra
Exemples reels	Firebase (Google), AWS Amplify, Supabase, Appwrite
Cas d'usage	Apps mobiles, MVPs, dashboards, apps no-backend
Avantage	Demarrage ultra-rapide, pas d'ops a gerer
Inconvenient	Moins de controle, vendor lock-in, couts variables

Simulation BaaS avec LocalStack

On simule un BaaS avec : DynamoDB (base NoSQL managee) + S3 (stockage fichiers) + IAM (authentication service) + CloudWatch (monitoring).
C'est exactement ce que AWS Amplify ou Firebase proposent en arriere-plan.

Exercice — Deployer un BaaS avec Terraform

Structure du projet

```
baas_project/  
├── main.tf  
├── dynamodb.tf      # Base de donnees NoSQL  
├── s3.tf             # Stockage fichiers  
├── iam.tf            # Accés securises  
├── outputs.tf  
└── app.py            # Client Python simulant une app mobile
```

dynamodb.tf — Base de donnees BaaS

```
# === dynamodb.tf ===  
  
# Table utilisateurs (equivalent Firebase Firestore)  
resource "aws_dynamodb_table" "users" {  
  name           = "baas-users"  
  billing_mode   = "PAY_PER_REQUEST" # auto-scaling, pas de capacite fixe  
  hash_key       = "userId"  
  
  attribute {  
    name = "userId"  
  }  
}
```

```

    type = "S" # S=String, N=Number, B=Binary
}

tags = { Service = "BaaS", ManagedBy = "terraform" }
}

# Table sessions (stockage authentication)
resource "aws_dynamodb_table" "sessions" {
    name           = "baas-sessions"
    billing_mode   = "PAY_PER_REQUEST"
    hash_key       = "sessionId"

    attribute { name = "sessionId", type = "S" }

    ttl {
        attribute_name = "expiresAt"
        enabled        = true # suppression automatique des sessions expirees
    }
}

```

s3.tf — Stockage fichiers BaaS

```

# === s3.tf ===

# Bucket stockage fichiers utilisateurs (photos, docs)
resource "aws_s3_bucket" "user_files" {
    bucket = "baas-user-files"
}

resource "aws_s3_bucket_versioning" "user_files" {
    bucket = aws_s3_bucket.user_files.id
    versioning_configuration { status = "Enabled" }
}

```

app.py — Client simulant une app mobile

```

# === app.py ===

import boto3, uuid, json
from datetime import datetime, timedelta

# Connexion LocalStack
config = dict(endpoint_url="http://localhost:4566",
               region_name="us-east-1",
               aws_access_key_id="test",
               aws_secret_access_key="test")

dynamo = boto3.resource("dynamodb", **config)
s3      = boto3.client("s3", **config)

def creer_utilisateur(nom, email):
    table = dynamo.Table("baas-users")
    user = {"userId": str(uuid.uuid4()), "nom": nom,
            "email": email, "creeA": datetime.now().isoformat()}
    table.put_item(Item=user)
    print(f"[BaaS] Utilisateur cree : {nom} ({user['userId'][:8]}...)")
    return user

def uploader_fichier(user_id, nom_fichier, contenu):
    cle = f"users/{user_id}/{nom_fichier}"
    s3.put_object(Bucket="baas-user-files", Key=cle, Body=contenu.encode())
    print(f"[BaaS] Fichier uploadé : {cle}")

```

```
if __name__ == "__main__":  
    u = creer_utilisateur("Alice Dupont", "alice@exemple.fr")  
    uploader_fichier(u["userId"], "profil.txt", "Donnees profil Alice")  
    print("[BaaS] Application BaaS operationnelle !")
```

```
terraform init && terraform apply -auto-approve  
python3 app.py  
awslocal dynamodb scan --table-name baas-users
```

```
[BaaS] Utilisateur cree : Alice Dupont (a3f7c2d1...)  
[BaaS] Fichier uploadé : users/a3f7c2d1.../profil.txt  
[BaaS] Application BaaS operationnelle !
```

Theorie — Qu'est-ce que le FaaS ?

Le FaaS (Function as a Service) est le modele 'serverless' le plus pousse : on deploye uniquement une fonction, pas un serveur. La fonction est executee a la demande, facturee a la milliseconde, et scaled automatiquement de 0 a l'infini.

Modele	Fonctionnement	Couts
Serveur traditionnel	Tourne en permanence	Couts fixes 24h/24
BaaS	Backend gere	Quelques services fixes
FaaS (serverless)	Execute a la demande	Facturation a l'invocation

Concept	Definition
AWS Lambda	FaaS AWS (supporte Python, Node, Java, Go)
Trigger	Ce qui declenche la fonction (HTTP, S3, DynamoDB, cron)
Cold start	Latence premiere invocation (la fonction se 'veille')
Timeout	Duree max d'execution (15 min pour Lambda)
Avantage	Zero gestion serveur, scaling automatique
Inconvenient	Cold start, stateless, debug plus complexe

Exercice — Deployer une Lambda avec Terraform

Etape 1 — La fonction Python (handler.py)

```
# === handler.py ===

import json
from datetime import datetime

def handler(event, context):
    """Fonction Lambda : traite un evenement et retourne une reponse"""
    print(f"[Lambda] Evenement reçu : {json.dumps(event)}")

    # Extraire les donnees de la requete
    nom = event.get("nom", "Inconnu")
    action = event.get("action", "saluer")

    if action == "saluer":
        message = f"Bonjour {nom} ! Il est {datetime.now().strftime('%H:%M')}:"
    elif action == "calculer":
        nb = event.get("nombre", 0)
        message = f"Le carre de {nb} est {nb**2}"
    else:
        message = f"Action inconnue : {action}"
```

```

    return {
        "statusCode": 200,
        "body": json.dumps({"message": message, "timestamp":
datetime.now().isoformat()})
    }

```

Etape 2 — lambda.tf : Deploiement Terraform

```

# === lambda.tf ===

# 1. Zipper la fonction (Terraform le fait automatiquement)
data "archive_file" "lambda_zip" {
    type          = "zip"
    source_file   = "handler.py"
    output_path   = "lambda.zip"
}

# 2. Role IAM pour la Lambda (moindre privilege)
resource "aws_iam_role" "lambda_role" {
    name = "faas-lambda-role"
    assume_role_policy = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Effect    = "Allow"
            Principal = { Service = "lambda.amazonaws.com" }
            Action     = "sts:AssumeRole"
        }]
    })
}

# 3. Policy : ecriture logs seulement
resource "aws_iam_role_policy" "lambda_logs" {
    name     = "lambda-logs-policy"
    role     = aws_iam_role.lambda_role.id
    policy   = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Effect    = "Allow"
            Action     = ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"]
            Resource   = "arn:aws:logs:*:*:*"
        }]
    })
}

# 4. La fonction Lambda
resource "aws_lambda_function" "ma_fonction" {
    function_name    = "faas-demo"
    filename         = data.archive_file.lambda_zip.output_path
    source_code_hash = data.archive_file.lambda_zip.output_base64sha256
    runtime          = "python3.12"
    handler          = "handler.handler" # fichier.fonction
    role             = aws_iam_role.lambda_role.arn
    timeout          = 30
    memory_size      = 128 # Mo

    environment {
        variables = { ENVIRONNEMENT = "localstack-dev" }
    }
}

# 5. Trigger S3 : la Lambda se declenche quand un fichier est uploadé
resource "aws_s3_bucket" "trigger_bucket" {
    bucket = "faas-trigger-bucket"
}

resource "aws_s3_bucket_notification" "trigger" {

```

```
bucket = aws_s3_bucket.trigger_bucket.id
lambda_function {
  lambda_function_arn = aws_lambda_function.ma_fonction.arn
  events               = ["s3:ObjectCreated:*"]
}
}
```

Etape 3 — Tester la Lambda

```
terraform init && terraform apply -auto-approve

# Invocation directe (event JSON)
awslocal lambda invoke \
  --function-name faas-demo \
  --payload '{"nom":"Bob","action":"calculer","nombre":7}' \
  response.json

cat response.json

# Invocation via trigger S3
echo "test" | awslocal s3 cp - s3://faas-trigger-bucket/nouveau-fichier.txt
awslocal lambda get-function --function-name faas-demo

{"statusCode": 200, "body": "{\"message\": \"Le carre de 7 est 49\", \"timestamp\": \"2025-01-21T10:30:00\"}"}

Lambda faas-demo : invoquee avec succes, cold start simule par LocalStack
```

Theorie — IAM et moindre privilege

IAM (Identity and Access Management) controle qui peut faire quoi sur quelle ressource. L'objectif est de donner a chaque utilisateur uniquement les droits dont il a reellement besoin — ni plus, ni moins.

Contexte du mini-projet 3

Votre collegue de classe doit pouvoir deployer son propre serveur (EC2) dans votre infrastructure. Il ne doit PAS pouvoir :

- Toucher vos donnees S3
- Modifier les regles IAM
- Supprimer des ressources qui ne lui appartiennent pas

Vous allez creer un compte IAM limite avec uniquement les droits EC2.

Exercice — Creer des acces securises avec Terraform

```
# === iam_collegue.tf ===

# — iam_collegue.tf —————

# 1. Groupe "developpeurs" (reutilisable pour plusieurs collegues)
resource "aws_iam_group" "developers" {
  name = "developers"
}

# 2. Policy : droit EC2 limite (pas de suppression VPC, pas d'IAM)
resource "aws_iam_policy" "dev_ec2_policy" {
  name        = "DevEC2LimitedPolicy"
  description = "Collegue peut deployer EC2 mais pas modifier IAM ni S3"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "AllowEC2Deploy"
        Effect   = "Allow"
        Action   = [
          "ec2:RunInstances",
          "ec2:DescribeInstances",
          "ec2:StartInstances",
          "ec2:StopInstances",
          "ec2:DescribeSecurityGroups",
          "ec2:CreateSecurityGroup",
          "ec2:AuthorizeSecurityGroupIngress"
        ]
        Resource = "*"
      },
      {
        Sid      = "AllowReadOwnLogs"
        Effect   = "Allow"
        Action   = ["logs:GetLogEvents", "logs:DescribeLogGroups"]
        Resource = "arn:aws:logs:*:*:log-group:/collegue/*"
      },
      {
        Sid      = "DenyIAMAndS3"
        Effect   = "Deny"
      }
    ]
  })
}
```

```

        Action = ["iam:*","s3:*","kms:*"] # JAMAIS acces IAM/S3/KMS
        Resource = "*"
    }
}
}))
}

# 3. Attacher la policy au groupe
resource "aws_iam_group_policy_attachment" "dev_policy" {
    group      = aws_iam_group.developers.name
    policy_arn = aws_iam_policy.dev_ec2_policy.arn
}

# 4. Compte utilisateur pour le collegue
resource "aws_iam_user" "colleague" {
    name = "colleague-dupont"
    tags = { Role = "developer", Projet = "cours-terraform" }
}

# 5. Ajouter le collegue dans le groupe
resource "aws_iam_user_group_membership" "colleague_dev" {
    user      = aws_iam_user.colleague.name
    groups    = [aws_iam_group.developers.name]
}

# 6. Cles d'accès programmatiques pour le collegue
resource "aws_iam_access_key" "colleague_key" {
    user = aws_iam_user.colleague.name
}

# 7. Output : afficher les credentials (à transmettre sécurisée)
output "colleague_access_key" {
    value     = aws_iam_access_key.colleague_key.id
    sensitive = false
}

output "colleague_secret_key" {
    value     = aws_iam_access_key.colleague_key.secret
    sensitive = true # masquée dans terraform plan
}

```

```

terraform apply -auto-approve

# Vérifier l'utilisateur et ses groupes
awslocal iam list-users
awslocal iam list-groups-for-user --user-name colleague-dupont

# Tester que le collegue ne peut PAS accéder à S3 (deny attendu)
awslocal iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::000000000000:user/colleague-dupont \
  --action-names "s3:GetObject" \
  --resource-arns "arn:aws:s3:::*"

awslocal iam list-groups-for-user --user-name colleague-dupont
=> Groups: [developers]

Test s3:GetObject => DENY (comme attendu - sécurité validée)
Test ec2:RunInstances => ALLOW (le collegue peut déployer son serveur)

```

A retenir : Bonne pratique IAM

Ne jamais donner les droits directement à un User — toujours passer par un Group.
 Utiliser des Deny explicites pour les ressources critiques (IAM, KMS, S3 sensible).
 Toujours inclure un Sid (identifiant) dans chaque Statement pour l'auditabilité.

Activer MFA pour tous les comptes humains (non simulateurs dans LocalStack).

Theorie — Block Storage vs Object Storage

Type	Service AWS	Cas d'usage ideal
Block Storage	EBS (AWS), disque SSD/HDD	Base de donnees, OS, fichiers acces aleatoire
Object Storage	S3 (AWS), Azure Blob	Fichiers statiques, backups, medias, logs
File Storage	EFS (AWS), NFS	Partage de fichiers entre instances

Critere	Block vs Object
Latence	< 1ms (block) vs 10-100ms (object)
Access pattern	Aleatoire byte par byte (block) vs fichier entier (object)
Scalabilite	Limitee par instance (block) vs infinie (object)
Cout 1 To/mois	~80 EUR (EBS SSD) vs ~23 EUR (S3 Standard)
Protocole	iSCSI, NVMe (block) vs HTTP REST (object)
Modification	In-place (block) vs remplacer l'objet entier (object)

Exercice — Benchmark et calcul de couts

```
# === storage.tf ===

# — storage.tf —————

# Object Storage : S3 (simulation)
resource "aws_s3_bucket" "benchmark_object" {
  bucket = "benchmark-object-storage"
}

# Block Storage simule : EBS via instance EC2
resource "aws_ebs_volume" "benchmark_block" {
  availability_zone = "us-east-1a"
  size              = 10    # 10 Go pour le test
  type              = "gp3"

  tags = { Name = "benchmark-block-storage" }
}
```

benchmark.py — Mesure des performances

```
# === benchmark.py ===
```

```

import boto3, time, os, tempfile

CONFIG = dict(endpoint_url="http://localhost:4566",region_name="us-east-1",
               aws_access_key_id="test",aws_secret_access_key="test")
s3 = boto3.client("s3", **CONFIG)

BUCKET = "benchmark-object-storage"
TAILLES = [1*1024, 100*1024, 10*1024*1024] # 1Ko, 100Ko, 10Mo

def bench_object_storage():
    print("\n=== OBJECT STORAGE (S3) ===")
    for taille in TAILLES:
        data = os.urandom(taille)
        cle = f"test-{taille}.bin"

        # ECRITURE
        debut = time.perf_counter()
        s3.put_object(Bucket=BUCKET, Key=cle, Body=data)
        t_ecriture = (time.perf_counter()-debut)*1000

        # LECTURE
        debut = time.perf_counter()
        s3.get_object(Bucket=BUCKET, Key=cle)
        t_lecture = (time.perf_counter()-debut)*1000

        print(f" {taille//1024:>6} Ko | Ecriture: {t_ecriture:7.2f}ms | Lecture: {t_lecture:7.2f}ms")

def bench_block_storage():
    """Simule un block storage avec fichiers locaux (equivalent EBS)"""
    print("\n=== BLOCK STORAGE (EBS simule - fichier local) ===")
    with tempfile.TemporaryDirectory() as tmpdir:
        for taille in TAILLES:
            data = os.urandom(taille)
            chemin = os.path.join(tmpdir, f"test-{taille}.bin")

            debut = time.perf_counter()
            with open(chemin,"wb") as f: f.write(data)
            t_ecriture = (time.perf_counter()-debut)*1000

            debut = time.perf_counter()
            with open(chemin,"rb") as f: f.read()
            t_lecture = (time.perf_counter()-debut)*1000

            print(f" {taille//1024:>6} Ko | Ecriture: {t_ecriture:7.2f}ms | Lecture: {t_lecture:7.2f}ms")

def calculer_couts_lto():
    print("\n=== COUT MENSUEL POUR 1 To ===")
    print(f" S3 Standard : ~23 EUR/mois (0.023 USD/Go)")
    print(f" S3 + requetes : ~25 EUR/mois (+ 0.004 USD/1000 requetes GET)")
    print(f" EBS gp3 : ~82 EUR/mois (0.08 USD/Go)")
    print(f" EBS io2 : ~125 EUR/mois (0.125 USD/Go)")
    print(f" => Economie S3 vs EBS gp3 : ~57 EUR/mois pour 1 To")

bench_object_storage()
bench_block_storage()
calculer_couts_lto()

```

```

=== OBJECT STORAGE (S3) ===
      1 Ko | Ecriture: 12.34ms | Lecture: 8.21ms
     100 Ko | Ecriture: 18.76ms | Lecture: 14.53ms
    10240 Ko | Ecriture: 87.45ms | Lecture: 61.22ms

=== BLOCK STORAGE (EBS simule - fichier local) ===

```

```
1 Ko | Ecriture: 0.08ms | Lecture: 0.04ms
100 Ko | Ecriture: 0.31ms | Lecture: 0.19ms
10240 Ko | Ecriture: 3.12ms | Lecture: 1.87ms
```

```
=== COUT MENSUEL POUR 1 To ===
```

```
S3 Standard : ~23 EUR/mois
```

```
EBS gp3 : ~82 EUR/mois
```

```
=> Economie S3 vs EBS gp3 : ~57 EUR/mois pour 1 To
```

Conclusion benchmark

Block Storage (EBS) : 100x plus rapide en latence — indispensable pour bases de données.

Object Storage (S3) : 3.5x moins cher — idéal pour fichiers statiques, backups, logs.

Règle pratique : si votre app modifie des bytes au milieu d'un fichier -> Block.

Si votre app lit/écrit des fichiers entiers -> Object.

Objectif pedagogique — Apprendre par l'exemple

Approche

Ce mini-projet volontairement deploye un bucket S3 NON securise, identifie les impacts potentiels, puis corrige chaque probleme. C'est la methode 'red team' : comprendre comment attaquer pour mieux defendre.

NE JAMAIS reproduire ce pattern en production ou sur un vrai compte AWS.

Exercice — Deployer et auditer un bucket non securise

Etape 1 — bucket_vulnerable.tf (intentionnellement mauvais)

```
# === bucket_vulnerable.tf ===

# !! CE CODE EST VOLONTAIREMENT NON SECURISE — USAGE PEDAGOGIQUE UNIQUEMENT !!

resource "aws_s3_bucket" "vulnerable" {
  bucket = "bucket-vulnerable-demo"
  # OUBLI 1 : pas de tags, pas de description
}

# OUBLI 2 : acces public NON bloque
# resource "aws_s3_bucket_public_access_block" ... { MANQUANT }

# OUBLI 3 : ACL publique (tout le monde peut lire)
resource "aws_s3_bucket_acl" "vulnerable_acl" {
  bucket = aws_s3_bucket.vulnerable.id
  acl    = "public-read" # !! DANGEREUX : tout internet peut lire
}

# OUBLI 4 : pas de chiffrement
# resource "aws_s3_bucket_server_side_encryption_configuration" ... { MANQUANT }

# OUBLI 5 : pas de versioning (pas de recuperation en cas d'effacement)
# resource "aws_s3_bucket_versioning" ... { MANQUANT }

# OUBLI 6 : politique permissive (tout le monde peut lire et lister)
resource "aws_s3_bucket_policy" "vulnerable_policy" {
  bucket = aws_s3_bucket.vulnerable.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = "*" # !! TOUT LE MONDE
      Action    = ["s3:GetObject", "s3:ListBucket"] # lecture + listage
      Resource  = ["arn:aws:s3:::bucket-vulnerable-demo",
                  "arn:aws:s3:::bucket-vulnerable-demo/*"]
    }]
  })
}
```

Etape 2 — Simuler une attaque et identifier les impacts

```

terraform apply -auto-approve

# Uploader des donnees sensibles (simulation)
echo "CONFIDENTIEL: mot de passe admin = SuperSecret123" | \
  awslocal s3 cp - s3://bucket-vulnerable-demo/config-secret.txt
echo "Numero carte: 4111111111111111" | \
  awslocal s3 cp - s3://bucket-vulnerable-demo/donnees-client.txt

# Un attaquant anonyme peut LISTER le bucket (acl public-read)
awslocal s3 ls s3://bucket-vulnerable-demo

# Un attaquant peut LIRE n'importe quel fichier
awslocal s3 cp s3://bucket-vulnerable-demo/config-secret.txt -

# L'attaquant voit la liste des fichiers :
2025-01-21 config-secret.txt
2025-01-21 donnees-client.txt

# L'attaquant lit le fichier confidentiel :
CONFIDENTIEL: mot de passe admin = SuperSecret123

IMPACT : Violation RGPD + compromission totale du systeme

```

Etape 3 — Tableau des vulnerabilites identifiees

Vulnerabilite	Impact	Correction Terraform
Acces public non bloque	Tout internet peut lister/lire le bucket	aws_s3_bucket_public_access_block
ACL public-read	Fichiers lisibles sans authentication	acl = private (default)
Politique trop permissive	Principal: * autorise tout le monde	Restreindre a des ARN specifiques
Pas de chiffrement	Donnees lisibles en clair si acces obtenu	aws_s3_bucket_server_side_encryption
Pas de versioning	Suppression irreversible des fichiers	aws_s3_bucket_versioning enabled
Pas de logging	Impossible de detecter une intrusion	aws_s3_bucket_logging

Etape 4 — bucket_securise.tf (version corrigee)

```

# === bucket_securise.tf ===

resource "aws_s3_bucket" "securise" {
  bucket = "bucket-securise-demo"
  tags = { Owner = "equipe-a", Classification = "confidentiel" }
}

# CORRECTION 1 : bloquer tout acces public
resource "aws_s3_bucket_public_access_block" "securise" {
  bucket = aws_s3_bucket.securise.id
  block_public_acls = true    block_public_policy = true
  ignore_public_acls = true  restrict_public_buckets = true
}

# CORRECTION 2 : chiffrement AES-256
resource "aws_s3_bucket_server_side_encryption_configuration" "securise" {
  bucket = aws_s3_bucket.securise.id

```

```
rule { apply_server_side_encryption_by_default { sse_algorithm = "AES256" } }  
}  
  
# CORRECTION 3 : versioning active  
resource "aws_s3_bucket_versioning" "securise" {  
  bucket = aws_s3_bucket.securise.id  
  versioning_configuration { status = "Enabled" }  
}
```

README.md — Ce que nous n'avons pas securise (a rediger)

Ce README documente les failles intentionnelles du bucket vulnérable.

1. Public Access Block manquant -> tout internet peut accéder au bucket.
2. ACL public-read -> les fichiers sont lisibles sans credentials.
3. Bucket Policy trop permissive (Principal: *) -> listage et lecture libres.
4. Chiffrement absent -> données en clair dans les data centers AWS.
5. Versioning absent -> suppression accidentelle ou malveillante irréversible.
6. Logging absent -> aucune trace des accès, impossible d'auditer une intrusion.

Theorie — Scaling horizontal vs vertical

Concept	Definition
Scaling vertical	Augmenter la puissance d'une instance (CPU, RAM)
Scaling horizontal	Ajouter des instances identiques (scale out)
Auto Scaling Group	Groupe d'instances qui s'ajuste automatiquement
Load Balancer	Repartit le trafic entre toutes les instances
Politique de scaling	Regles qui definissent quand ajouter/retirer des instances
Cooldown	Delai entre deux actions de scaling (evite l'oscillation)

```
# === scaling.tf ===

# — scaling.tf —————

# 1. Launch Template : configuration de chaque instance
resource "aws_launch_template" "app" {
  name_prefix   = "app-template-"
  image_id      = "ami-12345678"
  instance_type = "t3.micro"

  user_data = base64encode(<<-EOF
    #!/bin/bash
    # Script execute au demarrage de chaque instance
    apt-get update -y
    apt-get install -y python3
    python3 -m http.server 8080 &
    echo "Instance demarree : $(hostname)" > /tmp/status
  EOF
)

  vpc_security_group_ids = [aws_security_group.app_sg.id]

  tag_specifications {
    resource_type = "instance"
    tags = { Name = "app-instance", ManagedBy = "autoscaling" }
  }
}

# 2. Auto Scaling Group : min 1, max 5 instances
resource "aws_autoscaling_group" "app" {
  name           = "app-asg"
  min_size       = 1    # toujours au moins 1 instance
  max_size       = 5    # jamais plus de 5
  desired_capacity = 1   # demarre avec 1
  vpc_zone_identifier = [aws_subnet.public.id]

  launch_template {
    id       = aws_launch_template.app.id
    version = "$Latest"
  }
}
```

```

# Health check : retirer les instances malades
health_check_type      = "ELB"
health_check_grace_period = 60

tag {
  key      = "Name"
  value    = "app-asg-instance"
  propagate_at_launch = true
}
}

# 3. Politique de scale OUT (charge haute -> ajouter des instances)
resource "aws_autoscaling_policy" "scale_out" {
  name                  = "scale-out-cpu"
  autoscaling_group_name = aws_autoscaling_group.app.name
  adjustment_type       = "ChangeInCapacity"
  scaling_adjustment     = 2      # ajouter 2 instances
  cooldown              = 120    # attendre 2 min avant de rescaler
}

# 4. Alarme CloudWatch : CPU > 70% pendant 2 minutes -> scale out
resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name          = "cpu-high-70"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods  = 2
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 60
  statistic            = "Average"
  threshold            = 70

  dimensions = { AutoScalingGroupName = aws_autoscaling_group.app.name }
  alarm_actions = [aws_autoscaling_policy.scale_out.arn]
}

# 5. Politique de scale IN (charge faible -> retirer des instances)
resource "aws_autoscaling_policy" "scale_in" {
  name                  = "scale-in-cpu"
  autoscaling_group_name = aws_autoscaling_group.app.name
  adjustment_type       = "ChangeInCapacity"
  scaling_adjustment     = -1    # retirer 1 instance
  cooldown              = 300    # attendre 5 min
}

# 6. Alarme : CPU < 20% pendant 5 minutes -> scale in
resource "aws_cloudwatch_metric_alarm" "cpu_low" {
  alarm_name          = "cpu-low-20"
  comparison_operator = "LessThanThreshold"
  evaluation_periods  = 5
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 60
  statistic            = "Average"
  threshold            = 20

  dimensions = { AutoScalingGroupName = aws_autoscaling_group.app.name }
  alarm_actions = [aws_autoscaling_policy.scale_in.arn]
}

```

Simulation de charge et observation du scaling

```

terraform apply -auto-approve

# Verifier l'ASG
awslocal autoscaling describe-auto-scaling-groups --auto-scaling-group-names app-asg

```

```
# Simuler une charge CPU elevee
awslocal cloudwatch put-metric-data \
  --namespace "AWS/EC2" \
  --metric-name "CPUUtilization" \
  --dimensions "Name=AutoScalingGroupName,Value=app-asg" \
  --value 85 --unit Percent

# Observer le scaling (attendre ~2 min)
awslocal autoscaling describe-scaling-activities --auto-scaling-group-name app-asg

ScalingActivities:
- Description: Launching 2 new instances (CPU alarm: 85% > 70%)
  Status: Successful
  StartTime: 2025-01-21T10:05:00
  AutoScalingGroup: app-asg
  CurrentDesiredCapacity: 3 (etait 1, maintenant 3)
```

Diagramme du flux de scaling

Trafic entrant --> Load Balancer --> Instances (1 a 5)

CPU > 70% --> Alarme CloudWatch --> Scale OUT (+2 instances)

CPU < 20% --> Alarme CloudWatch --> Scale IN (-1 instance)

Instance malade --> Health Check ELB --> Remplacement automatique

Ce mini-projet reutilise l'application BaaS du projet 1 et y applique l'ensemble des mesures de securite : IAM, chiffrement, reseau, monitoring et backup avec PRA.

Architecture cible

BaaS (DynamoDB + S3) du projet 1

- + IAM roles avec moindre privilege (projet 3)
- + Chiffrement KMS en transit et au repos
- + Security Groups (reseau isole)
- + CloudWatch Logs + Alarmes + Dashboard
- + Backup automatique quotidien + Plan de restauration

PILIER 1 — IAM : Roles et moindre privilege

```
# === iam.tf ===

# — iam.tf —————

# Cle KMS : chiffrement centralise (partagee entre tous les piliers)
resource "aws_kms_key" "main" {
  description = "Cle principale securite application"
  enable_key_rotation = true
  tags = { ManagedBy = "terraform" }
}
resource "aws_kms_alias" "main" {
  name = "alias/app-securisee"
  target_key_id = aws_kms_key.main.key_id
}

# Role : application backend (acces DynamoDB + S3 lecture uniquement)
resource "aws_iam_role" "app_role" {
  name = "app-backend-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{ Effect="Allow", Principal={Service="lambda.amazonaws.com"},
    Action="sts:AssumeRole" }]
  })
}

resource "aws_iam_role_policy" "app_policy" {
  role = aws_iam_role.app_role.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      { Effect="Allow",
    Action=["dynamodb:GetItem","dynamodb:PutItem","dynamodb:Query"],
    Resource="arn:aws:dynamodb:*:*:table/baas-*" },
      { Effect="Allow", Action=["s3:GetObject","s3:PutObject"],
    Resource=["arn:aws:s3:::app-files-*","arn:aws:s3:::app-files-*/*"] },
      { Effect="Allow",
    Action=["logs:CreateLogGroup","logs:PutLogEvents","logs:CreateLogStream"],
    Resource="arn:aws:logs:*:*:log-group:/app/*" },
      { Effect="Allow", Action=["kms:Decrypt","kms:GenerateDataKey"],
    Resource=aws_kms_key.main.arn },
      { Sid="DenyAdmin", Effect="Deny",
    Action=["iam:*","s3:DeleteBucket","dynamodb:DeleteTable","kms:DeleteKey"],
    Resource="*" }
    ]
  })
}
```

```
    })  
  }  
}
```

PILIER 2 — Chiffrement en transit et au repos

```
# === encryption.tf ===  
  
# — encryption.tf —————  
  
# S3 chiffrer + acces public bloque + HTTPS force  
resource "aws_s3_bucket" "app_files" { bucket = "app-files-securises" }  
  
resource "aws_s3_bucket_public_access_block" "app_files" {  
  bucket = aws_s3_bucket.app_files.id  
  block_public_acls = true    block_public_policy = true  
  ignore_public_acls = true  restrict_public_buckets = true  
}  
  
resource "aws_s3_bucket_server_side_encryption_configuration" "app_files" {  
  bucket = aws_s3_bucket.app_files.id  
  rule {  
    apply_server_side_encryption_by_default {  
      sse_algorithm = "aws:kms"  
      kms_master_key_id = aws_kms_key.main.arn  
    }  
  }  
}  
  
# Policy HTTPS uniquement (chiffrement en transit)  
resource "aws_s3_bucket_policy" "force_https" {  
  bucket = aws_s3_bucket.app_files.id  
  policy = jsonencode({  
    Version = "2012-10-17"  
    Statement = [{  
      Effect="Deny", Principal="*", Action="s3:*",  
  
      Resource=["${aws_s3_bucket.app_files.arn}", "${aws_s3_bucket.app_files.arn}/*"],  
      Condition={ Bool={ "aws:SecureTransport"="false" } }  
    }]  
  })  
}
```

PILIER 3 — Network Security

```
# === network.tf ===  
  
# — network.tf —————  
  
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
  tags = { Name = "app-vpc" }  
}  
  
resource "aws_subnet" "private" {  
  vpc_id = aws_vpc.main.id    cidr_block = "10.0.2.0/24"  
  tags = { Name = "private-db", Tier = "database" }  
}  
  
# Security Group : application (HTTPS entrant, DB sortant)  
resource "aws_security_group" "app" {  
  name = "sg-app-securisee"  
  vpc_id = aws_vpc.main.id
```

```

    ingress { from_port=443, to_port=443, protocol="tcp"
               cidr_blocks=["0.0.0.0/0"], description="HTTPS uniquement" }

    egress { from_port=0, to_port=0, protocol="-1"
             cidr_blocks=["10.0.0.0/16"], description="Interne seulement" }
}

```

PILIER 4 — Monitoring, Logs et Dashboard

```

# === monitoring.tf ===

# — monitoring.tf —————

resource "aws_cloudwatch_log_group" "app" {
  name           = "/app/production"
  retention_in_days = 30
  kms_key_id     = aws_kms_key.main.arn
}

resource "aws_cloudwatch_log_group" "security" {
  name           = "/app/security"
  retention_in_days = 90 # audit longue duree
  kms_key_id     = aws_kms_key.main.arn
}

resource "aws_sns_topic" "alerts" {
  name           = "app-alerts"
  kms_master_key_id = aws_kms_key.main.id
}

# Alarme : erreurs application
resource "aws_cloudwatch_metric_alarm" "errors" {
  alarm_name           = "app-errors-high"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods   = 2   metric_name = "ErrorCount"
  namespace            = "App/Lambda"   period = 60
  statistic             = "Sum"   threshold = 10
  alarm_actions         = [aws_sns_topic.alerts.arn]
}

# Alarme : activite IAM suspecte
resource "aws_cloudwatch_metric_alarm" "iam_denied" {
  alarm_name           = "iam-access-denied"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods   = 1   metric_name = "AccessDeniedCount"
  namespace            = "CloudTrailMetrics"   period = 300
  statistic             = "Sum"   threshold = 5
  alarm_actions         = [aws_sns_topic.alerts.arn]
}

# Dashboard centralise
resource "aws_cloudwatch_dashboard" "app" {
  dashboard_name = "App-Security-Dashboard"
  dashboard_body = jsonencode({
    widgets = [
      { type="metric", properties={ title="Erreurs Lambda",
        metrics=[["App/Lambda","ErrorCount"]], period=60 } },
      { type="metric", properties={ title="Acces refuses IAM",
        metrics=[["CloudTrailMetrics","AccessDeniedCount"]], period=300 } },
      { type="log", properties={ title="Logs securite",
        query="fields @timestamp, @message | sort desc | limit 20",
        logGroupNames=["/app/security"] } }
    ]
  })
}

```

```
}
```

PILIER 5 — Backup automatique et PRA

```
# === backup.tf ===

# — backup.tf —————

# Bucket backup chiffre avec versioning et lifecycle
resource "aws_s3_bucket" "backup" { bucket = "app-backups-securises" }

resource "aws_s3_bucket_versioning" "backup" {
  bucket = aws_s3_bucket.backup.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "backup" {
  bucket = aws_s3_bucket.backup.id
  rule { apply_server_side_encryption_by_default {
    sse_algorithm = "aws:kms"    kms_master_key_id = aws_kms_key.main.arn
  }}
}

resource "aws_s3_bucket_lifecycle_configuration" "backup" {
  bucket = aws_s3_bucket.backup.id
  rule {
    id = "backup-lifecycle"    status = "Enabled"
    transition { days=30, storage_class="STANDARD_IA" }
    transition { days=90, storage_class="GLACIER" }
    expiration { days = 365 }
  }
}

# Vault AWS Backup centralise
resource "aws_backup_vault" "main" {
  name          = "app-backup-vault"
  kms_key_arn   = aws_kms_key.main.arn
}

resource "aws_backup_plan" "daily" {
  name = "DailyBackupPlan"
  rule {
    rule_name          = "backup-quotidien"
    target_vault_name  = aws_backup_vault.main.name
    schedule            = "cron(0 2 * * ? *)" # 2h chaque nuit
    lifecycle { delete_after = 30 }
  }
}
```

PLAN DE REPRISE D'ACTIVITE (PRA) — Mini Projet 7

1. Perimetre et objectifs

Parametre	Valeur
Application	BaaS securisee : DynamoDB + S3 + Lambda
RPO cible	1 heure (perte de donnees maximale acceptable)
RTO cible	4 heures (duree max avant remise en service)
Responsable	DevOps + DBA (equipe projet)
Frequence revue	Semestrielle ou apres tout incident
Outil de restauration	Terraform + script restore.sh

2. Scenarios et procedures de restauration

Scenario	RTO	RPO	Procedure
Suppression accidentelle S3	< 30min	< 1h	awslocal s3api list-object-versions → awslocal s3api copy-object (restaurer version)
Corruption DynamoDB	< 2h	< 1h	Restaurer depuis snapshot AWS Backup → valider l'integrite → relancer l'app
Cle KMS compromise	< 4h	0 (pas de perte)	Disable key → new key → re-chiffrer ressources → update policies → audit logs
Intrusion reseau	< 1h	0	Modifier Security Groups → invalider IAM keys → analyser CloudWatch → notifier RSSI
Panne totale infra	< 8h	< 1h	terraform destroy → terraform apply → restaurer backup → update DNS

3. Script de restauration automatise

```
# === restore.sh ===

#!/bin/bash
# restore.sh - PRA automatise
# Usage : ./restore.sh [scenario]

SCENARIO=$1
BUCKET_BACKUP="app-backups-securises"
log() { echo "[PRA $(date +%H:%M:%S)] $1"; }

case $SCENARIO in
    "s3-restore")
        log "DEBUT restauration S3"
        awslocal s3api list-object-versions --bucket app-files-securises
        log "Identifiez le VersionId a restaurer, puis :"
        log "awslocal s3api copy-object --copy-source BUCKET/KEY?versionId=ID ..."
        ;;
```

```

"infra-rebuild")
  log "DEBUT reconstruction infrastructure"
  terraform destroy -auto-approve
  terraform apply -auto-approve
  log "Infrastructure reconstruite"
;;
"network-lockdown")
  log "LOCKDOWN reseau - blocage urgence"
  awslocal ec2 revoke-security-group-ingress \
    --group-name sg-app-securisee --protocol tcp \
    --port 0-65535 --cidr 0.0.0.0/0
  log "Reseau isole"
;;
"kms-rotation")
  log "Rotation cle KMS suite compromission"
  # Desactiver ancienne cle
  awslocal kms disable-key --key-id alias/app-securisee
  # Creer nouvelle cle via Terraform
  terraform apply -target=aws_kms_key.main -auto-approve
  log "Nouvelle cle deployee - re-chiffrement necessaire"
;;
*)
  echo "Usage: $0 [s3-restore|infra-rebuild|network-lockdown|kms-rotation]"
  exit 1
;;
esac

```

4. Plan de secours (fallback)

Plan de secours si le PRA primaire echoue

Si terraform apply echoue pendant la restauration :

1. Rollback vers la derniere version stable du code Terraform (git checkout <tag>)
2. Restaurer manuellement les ressources critiques via awslocal CLI
3. Activer le mode maintenance (page statique S3) pendant la restauration

Si LocalStack est indisponible (en production AWS) :

1. Basculer vers la region secondaire (terraform workspace select dr)
2. Mettre a jour Route 53 pour pointer vers la region secondaire
3. Notifier les utilisateurs via la page de statut

Contact d'urgence : responsable DevOps -> DBA -> RSSI -> Direction

5. Planning des tests PRA

Test	Frequence	Procedure
Test restauration S3	Mensuel	Restaurer un fichier depuis version precedente, valider l'integrite
Rebuild complet infra	Trimestriel	terraform destroy + apply, mesurer temps vs RTO de 4h
Simulation intrusion	Semestriel	Simuler IP malveillante, verifier detection et lockdown
Exercice PRA complet	Annuel	Simuler perte totale, chronometrer chaque etape

RECAPITULATIF — Les 7 mini-projets

Projet	Ressources Terraform	Ce qu'on apprend
01 — BaaS	DynamoDB + S3 + IAM	Simuler Firebase/Amplify en local
02 — FaaS	Lambda + S3 trigger + IAM role	Fonction serverless avec declencheur S3
03 — IAM	Users, Groups, Policies, Deny explicite	Acces securise et auditable pour un collegue
04 — Stockage	EBS vs S3 + benchmark Python + couts	Block 100x plus rapide, Object 3x moins cher
05 — Vuln S3	ACL public + policy permissive + audit	Identifier et corriger 6 failles S3
06 — Scaling	ASG + Launch Template + CloudWatch Alarms	Scale out a 70% CPU, scale in a 20%
07 — Securite	IAM + KMS + SG + Logs + Backup + PRA	Application blindee + plan de reprise

Structure de dossiers recommandee

```
terraform_cours/  
├── 00_commun/          # main.tf avec provider LocalStack  
├── 01_baas/            # dynamodb.tf + s3.tf + iam.tf + app.py  
├── 02_faas/            # lambda.tf + handler.py  
├── 03_iam/             # iam_colleague.tf  
├── 04_stockage/        # storage.tf + benchmark.py  
├── 05_vulnerabilite/   # bucket_vulnerable.tf + bucket_securise.tf + README.md  
├── 06_scaling/         # scaling.tf + network.tf  
└── 07_securite/        # iam.tf + encryption.tf + network.tf + monitoring.tf  
                        # + backup.tf + restore.sh
```

Workflow pour chaque projet

```
cd 0X_projet/  
cp ../00_commun/main.tf .    # copier le provider commun  
terraform init                # une seule fois par dossier  
terraform plan                 # verifier avant de deployer  
terraform apply -auto-approve # deployer sur LocalStack  
# ... tester l'exercice ...  
terraform destroy             # nettoyer avant le projet suivant
```

Points cles a retenir

BaaS vs FaaS : BaaS = services geres, FaaS = fonctions a la demande, zero serveur.
IAM : toujours refuser d'abord, ouvrir au minimum, Deny > Allow.
Stockage : Block = latence ultra-faible (BDD), Object = cout bas (fichiers).
Securite S3 : bloquer l'accès public, chiffrer, forcer HTTPS, activer le versioning.
Scaling : Auto Scaling Group + Load Balancer = disponibilite et elasticite automatiques.
Securite complete : IAM + KMS + SG + Logs + Backup = defense en profondeur.
PRA : sans test regulier, le plan ne vaut rien — tester au minimum trimestriellement.